Management

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Purpose . . . 2
Source . . . 2
File format . . . 2
harv.ops . . . 2
graze.ops . . . 4
irr.ops . . . 6
chem_app.ops . . . 8
fire.ops . . . 10
sweep.ops . . . 12
Example . . . 14
Related files . . . 14
Purpose . . . 14
Source . . . 14
File format . . . 14
Example . . . 15
Related files . . . 16
Purpose . . . 16
Source . . . 16
Format . . . 16
Example . . . 19
Related files . . . 19
Purpose . . . 19
Source . . . 19
Format . . . 19
Example . . . 21
manure_db.frt . . . 21
manure_om.frt . . . 22
Related files . . . 23
Purpose . . . 23
Source . . . 23
Format . . . 23
Example . . . 25
Related files . . . 25
What replaces it . . . 25
Migration . . . 25
Related files . . . 26
Purpose . . . 26
Source . . . 26
Format . . . 26
Example . . . 26
Related files . . . 27
Purpose . . . 27
Source . . . 27
Format . . . 27

Example . . . 28
Related files . . . 28
management.sch . . . 28
puddle.ops . . . 30
transplant.plt . . . 32
co2.out and co2_yr.dat . . . 34
object.prt . . . 34

Purpose

The six `.ops` files declared in `type input_ops` are parameter databases for the management operations that `management.sch` and the `lum.dtl` decision tables can invoke. Each file is a named list of parameter sets that other files reference by `name`.

All six readers follow the same structure: skip two header lines, count records, allocate, rewind, skip two, read one record per row into the corresponding `*_db` array.

Source

- Readers: `src/mgt_read_harvops.f90`, `src/mgt_read_grazeops.f90`, `src/mgt_read_irrops.f90`, `src/mgt_read_chemapp.f90`, `src/mgt_read_fireops.f90`, `src/mgt_read_sweepops.f90`.
- Type definitions: `src/mgt_operations_module.f90`.

File format

For every file:

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3 and following: one record per operation. List-directed read. The reader counts records on a first pass, allocates the database array, rewinds, and reads the values.

The "Common" first column is the operation name; other files reference it.

harv.ops

Reader: `mgt_read_harvops.f90` → `harvop_db` (`type harvest_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	typ	char(40)	“”	none	`grain`, `biomass`, `residue`, `tree`, or `tuber`
3	hi_ovr	real	0	none	harvest index override at harvest
4	eff	real	0	fraction	harvest efficiency. Fraction of harvested yield removed; the remainder is left as residue
5	bm_min	real	0	kg/ha	minimum biomass required to allow harvest

graze.ops

Reader: `mgt_read_grazeops.f90` → `grazeop_db` (`type grazing_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	fertnm	char(40)	“”	none	manure name pointing into `fertilizer.frt`
3	manure_id	int	0	none	resolved id of `fertnm` in `fertilizer.frt`
4	eat	real	0	(kg/ha)/day	dry biomass removed by grazing per day
5	tramp	real	0	(kg/ha)/day	dry biomass removed by trampling per day
6	manure	real	0	(kg/ha)/day	dry manure deposited per day
7	biomin	real	0	kg/ha	minimum plant biomass to allow grazing

irr.ops

Reader: `mgt_read_irrops.f90` → `irrop_db` (`type irrigation_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	amt_mm	real	25.4	mm	application amount
3	eff	real	0	none	in-field application efficiency
4	surq	real	0	fraction	surface runoff ratio
5	dep_mm	real	0	mm	depth of application for subsurface irrigation
6	salt	real	0	mg/kg	total salt concentration in irrigation water
7	no3	real	0	mg/kg	nitrate concentration
8	po4	real	0	mg/kg	phosphate concentration

chem_app.ops

Reader: `mgt_read_chemapp.f90` → `chemapp_db` (`type chemical_application_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	form	char(40)	“”	none	`solid` or `liquid`
3	op_typ	char(40)	“”	none	`spread`, `spray`, `inject`, or `direct`
4	app_eff	real	0	none	application efficiency
5	foliar_eff	real	0	none	foliar efficiency
6	inject_dep	real	0	mm	injection depth
7	surf_frac	real	0	fraction	fraction in upper 10 mm of soil
8	drift_pot	real	0	none	drift potential
9	aerial_unif	real	0	none	aerial uniformity

fire.ops

Reader: `mgt_read_fireops.f90` → `fire_db` (`type fire_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	cn2_upd	real	0	none	change applied to SCS curve number II value
3	fr_burn	real	0	fraction	fraction of cover burned

sweep.ops

Reader: `mgt_read_sweepops.f90` → `sweepop_db` (`type streetsweep_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	eff	real	0	none	removal efficiency of the sweeping operation
3	fr_curb	real	0	none	availability factor: fraction of curb length that is sweepable

Example

`refdata/Ames_sub1/harv.ops`:

```
harv.ops written by SWAT+ editor v2.3.3 on 2023-10-25
name                  typ                hi_ovr        eff        bm_min
harv_grain            grain              0.0           1.0        0.0
harv_residue          residue            0.0           1.0        0.0
```

The other five files follow the same shape with their own column sets.

Related files

- `landuse.lum` and `management.sch` reference operations by `name`.
- `*.dtl` decision tables can fire operations by `name`.
- `fertilizer.frt` provides the `manure_id` resolution target for `graze.ops`.

Purpose

A decision-table file holds one or more tables. Each table evaluates a set of conditions every time step and fires the matching action when an alternative is satisfied. SWAT+ uses decision tables to drive land-use management (planting, harvest, fertiliser, irrigation), reservoir releases, scenario land-use changes, and flow control.

The four decision-table files declared in `type input_condition` are:

- `lum.dtl`: land-use management actions.
- `res_rel.dtl`: reservoir release rules.
- `scen_lu.dtl`: scenario-driven land-use changes.
- `flo_con.dtl`: flow control rules.

All four use the same file format. They differ only in which decision-table array they fill (`dtbl_lum`, `dtbl_res`, `dtbl_scen`, `dtbl_flo`).

Source

- Readers: `src/dtbl_lum_read.f90`, `src/dtbl_flocon_read.f90` (and matching readers for reservoir-release and scenario tables).
- Type definitions: `src/conditional_module.f90`. Each table is a `decision_table` instance; its `cond`, `alt`, and `act` arrays hold conditions, alternatives, and actions.

File format

A single file holds `mdtbl` tables.

- Line 1: title (skipped).
- Line 2: `mdtbl`, the number of decision tables in the file.
- Line 3: blank line (skipped).

For each of the `mdtbl` tables:

- One header line (skipped).
- One name line: `name`, `conds`, `alts`, `acts` (table name, number of conditions, number of alternatives, number of actions).
- One header line for the conditions block (skipped).
- `conds` lines, one per condition. Each line names a conditional variable plus `alts` alternative thresholds. See Condition row below.
- One header line for the actions block (skipped).
- `acts` lines, one per action. See Action row below.

Condition row

Each condition row is read into a `condition_var` record with the following fields, then `alts` alternative thresholds are read into the `alt(ic, 1..alts)` array.

Fields on a condition row:

#	Field	Type	Description
1	var	char	conditional variable name (for example `land_use`, `plant_gro`, `phu_plant`, `soil_water`, `prob_unif`)
2	obj	char	object the variable applies to
3	obj_num	int	object number
4	lim_var	char	limit-variable name
5	lim_op	char	comparison operator
6	lim_const	real	comparison constant
7+	alt(1..alts)	char	one alternative threshold per alternative column

When `var == "prob_unif"`, the reader rereads the row and treats the second token as the probability fraction `frac_app` rather than the standard condition layout.

Action row

#	Field	Type	Description
1	act_typ	char	action type (for example `plant`, `harvest`, `fert`, `irrigate`, `lu_change`)
2	obj	char	target object
3	obj_num	int	target object number
4	name	char	action name pointing into the matching ops or input database
5	option	char	option tag
6	const	real	numeric constant for the action
7	const2	real	second numeric constant
8	fp	char	flag or pointer
9+	act_hit(1..alts)	char	one outcome flag per alternative column

Notes

- The reader allocates `cond(conds)`, `alt(conds, alts)`, `act(acts)`, and `act_hit(alts)` based on the header values. A mismatch between declared counts and actual rows truncates or overruns these arrays.
- After all tables are read, `lum.dtl` does a second pass to count HRUs whose `land_use_mgt_c` matches any `land_use` condition. The counts feed probabilistic operations.

Example

Top of `refdata/Ames_sub1/lum.dtl`:

```
lum.dtl: written by SWAT+ editor for SWAT+ rev.60.5.7
8

corn_grain
name                 conds   alts   acts
pl_hv_corn               2      2      2
var                   obj   obj_num   lim_var   lim_op   lim_const   alt1   alt2
plant_gro           plant         1   tree      >        0           y      n
phu_plant           plant         1                               >  1.0    n      y
act_typ             obj   obj_num   name                 option   const   const2   fp   outcome1   outcom
plant               hru   0         corn                 below    0       0        y    y          n
harvest_kill        hru   0         harv_grain_corn      now      0       0        y    n          y
```

Reading this:

- File holds 8 tables. The first is named `pl_hv_corn` with 2 conditions, 2 alternatives, 2 actions.
- Condition 1 checks `plant_gro` on plant object 1 against `tree > 0`. Alt 1 fires when `y`, alt 2 fires when `n`.
- Condition 2 checks `phu_plant > 1.0`. Alt 1 fires when `n`, alt 2 fires when `y`.
- Action 1: `plant` HRU operation named `corn` with option `below`. Fires under alt 1, not alt 2.
- Action 2: `harvest_kill` HRU operation named `harv_grain_corn` with option `now`. Fires under alt 2, not alt 1.

The exact column layout varies a little between datasets because the reader is fully list-directed.

Related files

- `landuse.lum`. Plant and management names referenced by actions.
- `*.ops`. Operation parameter databases that the actions resolve `name` against.
- `management.sch`. Fixed schedules used alongside decision tables.

Purpose

`plants.plt` holds one parameter set per plant species the model can grow. Each record fills a `plant_db` entry in the `pldb` array. Plant growth, phenology, biomass-to-energy conversion, root and canopy geometry, nutrient uptake, and residue partition fractions all come from this file. HRU plant communities (`plant.ini`) and management operations (`plant.ops`, `landuse.lum`) reference these records by name.

Source

- Reader: `src/plant_parm_read.f90`
- Type definition: `src/plant_data_module.f90`, `type plant_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per plant. List-directed read into `pldb(ic)` (`type plant_db`). The last sub-object is `res_part_fracs` (`type residue_partition_fracs`), which contributes three trailing fields.

The reader has a conditional branch:

```
if (bsn_cc%nam1 == 0) then
  read (104,*,iostat=eof) pldb(ic)
else
  read (104,*,iostat=eof) pldb(ic), pl_class(ic)
end if
```

`bsn_cc%nam1` is a switch in `codes.bsn`. With the default `nam1 = 0`, the standard column list below applies. With `nam1 /= 0`, one extra `pl_class` token (a 25-character name like `row_crop`, `tree`, `grass`) is read at the end of each record.

#	Field	Type	Units	Default	Description
1	plantnm	char(40)	none	“”	plant name (used as the key from other files)
2	typ	char(18)	none	“”	plant category: `warm_annual`, `cold_annual`, `warm_annual_tuber`, `cold_annual_tuber`, `perennial`
3	trig	char(18)	none	“”	phenology trigger: `moisture_gro` or `temp_gro`
4	nfix_co	real	none	0.0	nitrogen fixation coefficient (0.5 for legumes, 0 for non-legumes)
5	days_mat	int	days	110	days to maturity. If 0, heat units run the entire growing season
6	bio_e	real	(kg/ha)/(MJ/m^2)	15.0	biomass-energy ratio
7	hvsti	real	(kg/ha)/(kg/ha)	0.76	harvest index: crop yield over above-ground biomass
8	blai	real	none	5.0	maximum potential leaf area index
9	frgrw1	real	fraction	0.05	fraction of growing season at the 1st point on the optimal LAI curve
10	laimx1	real	fraction	0.05	fraction of maximum LAI at the 1st point on the optimal LAI curve
11	frgrw2	real	fraction	0.4	fraction of growing season at the 2nd point on the optimal LAI curve
12	laimx2	real	fraction	0.95	fraction of maximum LAI at the 2nd point on the optimal LAI curve
13	dlai	real	fraction	0.99	fraction of growing season when LAI starts to decline
14	dlai_rate	real	none	1.0	exponent governing LAI decline rate
15	chtmx	real	m	6.0	maximum canopy height
16	rdmx	real	m	3.5	maximum root depth
17	t_opt	real	deg C	30.0	optimal temperature for plant growth
18	t_base	real	deg C	10.0	minimum (base) temperature for plant growth
19	cnyld	real	kg N / kg yield	0.0015	fraction of nitrogen in yield
20	cpyld	real	kg P / kg yield	0.0003	fraction of phosphorus in yield
21	pltnfr1	real	kg N / kg biomass	0.006	nitrogen uptake parameter 1 (emergence)
22	pltnfr2	real	kg N / kg biomass	0.002	nitrogen uptake parameter 2 (mid-season)
23	pltnfr3	real	kg N / kg biomass	0.0015	nitrogen uptake parameter 3 (maturity)
24	pltpfr1	real	kg P / kg biomass	0.0007	phosphorus uptake parameter 1 (emergence)
25	pltpfr2	real	kg P / kg biomass	0.0004	phosphorus uptake parameter 2 (mid-season)
26	pltpfr3	real	kg P / kg biomass	0.0003	phosphorus uptake parameter 3 (maturity)
27	wsyf	real	(kg/ha)/(kg/ha)	0.01	lower bound of harvest index under water stress
28	usle_c	real	none	0.001	minimum value of the USLE C factor for water erosion
29	gsi	real	m/s	0.002	maximum stomatal conductance
30	vpdfr	real	kPa	4.0	vapor pressure deficit at which `gmaxfr` is valid
31	gmaxfr	real	none	0.75	fraction of maximum stomatal conductance achieved at `vpdfr`
32	wavp	real	none	8.0	rate of decline in radiation use efficiency per unit VPD

#	Field	Type	Units	Default	Description
33	co2hi	real	uL CO2 / L air	660.0	elevated CO2 concentration at the 2nd point on the RUE-CO2 curve
34	bioehi	real	(kg/ha)/(MJ/m^2)	16.0	biomass-energy ratio at the elevated CO2 level (`co2hi`)
35	rsdco_pl	real	none	0.05	plant residue decomposition coefficient
36	alai_min	real	$m^{2/m}2$	0.75	minimum LAI during winter dormant period
37	laixco_tree	real	none	0.3	coefficient used to estimate maximum LAI during tree growth
38	mat_yrs	int	years	10	years to maturity (forced to at least 1 by the reader)
39	bmx_peren	real	metric tons/ha	1000.0	maximum biomass for forest
40	ext_coef	real	none	0.65	light extinction coefficient
41	leaf_tov_min	real	months	12.0	perennial leaf turnover rate with minimum stress
42	leaf_tov_max	real	months	3.0	perennial leaf turnover rate with maximum stress
43	bm_dieoff	real	fraction	0.0	above-ground biomass that dies off at dormancy
44	rsr1	real	fraction	0.0	initial root-to-shoot ratio at the beginning of the growing season
45	rsr2	real	fraction	0.0	root-to-shoot ratio at the end of the growing season
46	pop1	real	plants/m^2	0.0	plant population at the 1st point on the population-LAI curve
47	frlai1	real	fraction	0.0	fraction of maximum LAI at the 1st point on the population-LAI curve
48	pop2	real	plants/m^2	0.0	plant population at the 2nd point on the population-LAI curve
49	frlai2	real	fraction	0.0	fraction of maximum LAI at the 2nd point on the population-LAI curve
50	frsw_gro	real	fraction	0.5	30-day sum of P-PET that triggers monsoon-season growth in tropical plants
51	aeration	real	none	0.2	aeration stress factor
52	rsd_pctcov	real	none	0.0	residue factor for the percent cover equation
53	rsd_covfac	real	none	0.0	residue factor for the surface cover (C factor) equation
54	res_part_fracs%meta_frac	real	fraction	0.85	metabolic fraction of residue
55	res_part_fracs%str_frac	real	fraction	0.15	structural fraction of residue
56	res_part_fracs%lig_frac	real	fraction	0.12	lignin fraction of residue

When `cswat = 2` in `codes.bsn`, the three residue partition fractions are reinterpreted into above- and below-ground metabolic, structural, and lignin pools (`cswat_1_part_fracs`) in `plant_parm_read`. When `cswat /= 2`, the reader overwrites the read values with hard-coded defaults of 0.85, 0.15, and 0.12.

Example

First two records of `refdata/Ames_sub1/plants.plt`:

```
plants.plt
name  plnt_typ  gro_trig  nfix_co  days_mat  bm_e  harv_idx  lai_pot  frac_hu1  lai_max1  frac_hu2  lai_ma
alfa  perennial  temp_gro  0.5  0  20  0.9  4  0.15  0.01  0.5  0.95  0.9  0.5  0.9  3  20  4  0.02  0.003
cana  warm_annual  temp_gro  0  100  34  0.3  4.5  0.15  0.02  0.45  0.95  0.5  0.3  1.3  1.4  21  5  0.03
```

Notes on the example:

- Header column names written by the editor (`plnt_typ`, `gro_trig`, `bm_e`, `harv_idx`, `lai_pot`, `frac_hu1`, `frac_n_yld`, `frac_n_em`, `stcon_max`, `frac_stcon`, `ru_vpd`, `co2_hi`, `bm_e_hi`, `plnt_decomp`, `lai_min`, `bm_tree_acc`, `yrs_mat`, `bm_tree_max`, `ext_co`, `leaf_tov_mn`, `leaf_tov_mx`, `rt_st_beg`, `rt_st_end`, `plnt_pop1`, `frac_lai1`, `frac_sw_gro`, `avg_lig_frac`, `ab_lig_frac`, `bg_lig_frac`) do not match the field names in the source. The reader uses position only.
- The trailing `description` column is not part of `plant_db`. List-directed reads stop after the type is filled, so the description is silently discarded.
- For `alfa`, `days_mat = 0` is the explicit "use heat units for the full season" signal.

Related files

- `codes.bsn`. `bsn_cc%nam1` selects whether `pl_class` is read after each record. `bsn_cc%cswat` selects how residue partition fractions are interpreted.
- `plant.ini`. Lists which plants are present in each HRU at the start of simulation, keyed by `plantnm`.
- `landuse.lum` and `plant.ops`. Reference plant names from this file in management schedules.

Purpose

`fertilizer.frt` lists every fertilizer product the simulation can apply, with its mass fractions of mineral nitrogen, mineral phosphorus, organic nitrogen, organic phosphorus, and the share of mineral N that is in the NH3 form. Each record fills one `fertilizer_db` entry in the `fertdb` array. Fertilizer operations (in `chem_app.ops` and the management decision tables) reference entries here by name.

Source

- Reader: `src/fert_parm_read.f90`
- Type definition: `src/fertilizer_data_module.f90`, `type fertilizer_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per fertilizer product. List-directed read into `fertdb(it)` (`type fertilizer_db`).

#	Field	Type	Units	Default	Description
1	fertnm	char(16)	none	“”	fertilizer name (key referenced by `chem_app.ops` and management decision tables)
2	fminn	real	kg min-N / kg fertilizer	0.0	fraction of fertilizer that is mineral N (NO3 + NH3)
3	fminp	real	kg min-P / kg fertilizer	0.0	fraction of fertilizer that is mineral P
4	forgn	real	kg org-N / kg fertilizer	0.0	fraction of fertilizer that is organic N
5	forgp	real	kg org-P / kg fertilizer	0.0	fraction of fertilizer that is organic P
6	fnh3n	real	kg NH3-N / kg min-N	0.0	fraction of mineral N content that is NH3

The five real fractions must sum to less than or equal to 1 for a single product. `fminn + fminp + forgn + forgp` is the total nutrient mass per unit fertilizer; the remainder is carrier mass.

Example

First lines of `refdata/Ames_sub1/fertilizer.frt`:

```
fertilizer.frt: written by SWAT+ editor v2.2.2 on 2023-10-17 14:43 for SWAT+ rev.60.5.4
name                      min_n         min_p         org_n         org_p         nh3_n         pathogens  
elem_n                  1.00000       0.00000       0.00000       0.00000       0.00000              null  
elem_p                  0.00000       1.00000       0.00000       0.00000       0.00000              null  
anh_nh3                 0.82000       0.00000       0.00000       0.00000       1.00000              null  
urea                    0.46000       0.00000       0.00000       0.00000       1.00000              null  
46_00_00                0.46000       0.00000       0.00000       0.00000       0.00000              null  
18_46_00                0.18000       0.20200       0.00000       0.00000       0.00000              null  
dairy_fr                0.00700       0.00500       0.03100       0.00300       0.99000              null  
```

Notes on the example:

- The editor writes two trailing columns (`pathogens` and `description`) that are not in `fertilizer_db`. List-directed reads stop once the type is filled, so both trailing tokens are discarded by the reader. The literal `null` in the `pathogens` column has no effect.
- For `urea`, `fminn = 0.46` and `fnh3n = 1.00`. That is, 46% of the product is mineral N, and 100% of that mineral N is in the NH3 form.
- For `18_46_00`, `fminn = 0.18` and `fminp = 0.202`. Note that the third value in the "46" position is the P (P2O5 converted to P), not P2O5 directly.

manure_db.frt

Named manure types. Each entry points to one organic-matter record in `manure_om.frt` and (optionally) to records for pesticides, pathogens, heavy metals, salts, and other constituents that may be carried with the manure. This is the lookup used by manure-application operations and by manure-allocation.md.

Source

- Reader: src/manure_db_read.f90
- Type: src/fertilizer_data_module.f90, `type manure_database` (variable `manure_db`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per manure type. Two passes: count rows, then read.

#	Field	Type	Description
1	name	char(25)	Manure-type name. Referenced by manure-application operations and by `manure_allo.mnu`.
2	org_min	char(25)	Crosswalk into `manure_om.frt`. Resolved into `iorg_min`.
3	pests	char(25)	Pesticide profile name. Pointer field `ipests` is reserved (the crosswalk loop is commented out in the reader). Use `null` when not applicable.
4	paths	char(25)	Pathogen profile name. Pointer `ipaths`.
5	hmets	char(25)	Heavy-metal profile name. Pointer `imets`.
6	salts	char(25)	Salt-ion profile name. Pointer `isalts`.
7	constit	char(25)	Other-constituent profile name. Pointer `iconstit`.
8	descrip	char(80)	Free-text description.

After reading each row, the reader crosswalks `org_min` against `manure_om(:)%name` and stores the matching index in `manure_db(it)%iorg_min`. The other crosswalks are present in the type but not wired in this reader.

Example

`refdata/Osu_1hru/manure_db.frt`:

```
manure_db.frt: manure database
name          org_min            pests  paths  hmets  salts  constit  description
beef_salt     mw_beef_liq        null   null   null   null   null     midwest_beef_liquid_salt
```

One manure type, pointing at `mw_beef_liq` in `manure_om.frt`. No pesticide, pathogen, heavy-metal, salt, or constituent profile.

manure_om.frt

Organic-matter attributes for each manure type. Holds the water fraction and the C, mineral N, mineral P, organic N, organic P, and NH3 fractions used when a manure application is partitioned into the soil pools.

Source

- Reader: src/manure_orgmin_read.f90
- Type: src/fertilizer_data_module.f90, `type manure_attributes` (variable `manure_om`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per manure organic-matter profile.

#	Field	Type	Units	Description
1	name	char(64)	none	Profile name. Used as the crosswalk target from `manure_db.frt`.
2	frac_water	real	kg water / (kg manure + kg water)	Water fraction of the as-applied manure.
3	fcbn	real	kg C / kg manure	Carbon fraction.
4	fminn	real	kg min-N / kg manure	Mineral N fraction (NO3 + NH3).
5	fminp	real	kg min-P / kg manure	Mineral P fraction.
6	forgn	real	kg org-N / kg manure	Organic N fraction.
7	forgp	real	kg org-P / kg manure	Organic P fraction.
8	fnh3n	real	kg NH3-N / kg min-N	Share of mineral N that is NH3.
9	description	char(64)	none	Free-text description.

Example

`refdata/Osu_1hru/manure_om.frt`:

```
manure_om.frt: manure organic matter database
name           frac_water   fcbn          fminn         fminp         forgn         forgp         fnh3n  c
mw_beef_liq    0.988        0.000780576   0.000311751   0.00081295    0.002110312   0.000365707   0.99   r
mw_beef_slur   0.926        0.016786571   0.002697842   0.00081295    0.002110312   0.000365707   0.99   r
```

Two profiles. The liquid form is nearly all water (98.8%); the slurry form is 92.6% water with much higher solids and carbon.

Related files

- `chem_app.ops`. Fertilizer applications in management schedules reference `fertnm` from `fertilizer.frt` or a manure name from `manure_db.frt`.
- `manure-allocation.md`. Allocates manure from named source objects to HRUs and other receivers.
- `codes.bsn`. The active carbon model (`cswat`) controls how N from fertilizer and manure enters the soil pools.

Purpose

`tillage.til` defines each named tillage operation. Each record fills one `tillage_db` entry in the `tilldb` array. Tillage operations referenced in `chem_app.ops` and management decision tables look up the implement by name to get the mixing efficiency, mixing depth, random roughness, and ridge geometry that drive soil-mixing and surface-roughness updates in `mgt_newtillmix_*` and `mgt_tillfactor.f90`.

The reader also scans for the entry named `biomix`. If present, its `effmix` and `deptil` are copied into the module-level `bmix_eff` and `bmix_depth`, which control the daily biological mixing computed in `mgt_biomix.f90`. If no `biomix` entry is present, `bmix_eff` defaults to 0.2 and `bmix_depth` defaults to 50 mm.

Source

- Reader: `src/till_parm_read.f90`
- Type definition: `src/tillage_data_module.f90`, `type tillage_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per tillage operation. List-directed read into `tilldb(itl)` (`type tillage_db`).

#	Field	Type	Units	Default	Description
1	tillnm	char(16)	none	“”	tillage operation name (key from management schedules). The literal name `biomix` sets the biological mixing parameters at the module level
2	effmix	real	none	0.0	mixing efficiency of the tillage operation (fraction of soil mass redistributed within the till depth)
3	deptil	real	mm	0.0	depth of mixing caused by the tillage operation
4	ranrns	real	mm	0.0	random roughness imparted to the soil surface
5	ridge_ht	real	mm	0.0	ridge height
6	ridge_sp	real	mm	0.0	ridge interval (or row spacing)

Example

First lines of `refdata/Osu_1hru/tillage.til` (which has the title and header lines the reader expects):

```
tillage.til: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    mix_eff        mix_dp         rough      ridge_ht      ridge_sp  description
puddle                  0.980       150.000        00.000         0.000         0.000    rice_paddy_puddlin
```

The trailing `description` column is not in `tillage_db`. List-directed reads stop after the six fields are filled, so the description is silently discarded.

The Ames database has many more operations (the first records, after a correct two-line header, are):

```
chiselplow              0.65          225           20              0             0     chiselplow
cltiweed                0.3           100           15              0             0     cultiweeder
moldboardplow           0.95          250           30              0             0     mldboard
biomix                  0.01           50            0              0             0     biological_mixing
```

The `biomix` row above sets `bmix_eff = 0.01` and `bmix_depth = 50` at the module level after the file is read.

Related files

- `chem_app.ops` and `lum.dtl` (and similar decision tables). Tillage operations reference `tillnm` from this file.
- codes.bsn. The active carbon model (`cswat`) selects which `mgt_newtillmix_*` routine performs the soil pool mixing.
- `hru-data.hru` and `landuse.lum`. Identify the HRUs that receive the tillage operations defined here.

Removed

`carb_coefs.cbn` was removed. The keyword-style reader (`carbon_coef_read.f90`) and the dead companion file `basins_carbon.tes` have also been deleted. The 22 daily hard-coded coefficient overrides that previously lived in `cbn_zhang2.f90` are gone; user-supplied values now flow end-to-end.

What replaces it

Carbon inputs have been split into two flat-table files that follow the same layout convention as the rest of the SWAT+ inputs (title line, header line, data rows):

- carbon.bsn: 28 basin-wide scalar parameters in a single data row. Initial pool fractions, water coupling, manure partitioning, temperature and water factor selectors, the `t_cbn_*` triple, biomixing and tillage curves, photodegradation, residue C:N / C:P controls, and the slow-humus initialization method (`mathers_method`).
- carbon_lyr.bsn — per-layer rate constants and CO2 allocation fractions. One row per layer group (1 = top, 2 = subsurface).

Both files are required when `bsn_cc%cswat == 2` (the dynamic carbon model; the activation value moved from 1 to 2). Either missing or malformed file triggers `error stop`; there is no silent fallback to compiled defaults.

Migration

Old keyword in `carb_coefs.cbn`	New location
`cbn_diagnostics`	retired; output gating now lives in print.prt via the `hru_cb_*` and `lsu_cb_*` rows
`hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp`	per-layer rows of carbon_lyr.bsn
`a1co2`, `asco2`, `apco2`, `abco2`	per-layer rows of carbon_lyr.bsn
`org_frac` (`frac_seq`, `frac_hum_microb`, `frac_hum_slow`, `frac_hum_passive`)	`init_seq`, `init_microb`, `init_slow`, `init_passive` columns of carbon.bsn
`prmt_21`	`koc_c` column of carbon.bsn
`prmt_44`	`solc_ratio` column of carbon.bsn
`till_eff_days`	`till_eff_days` column of carbon.bsn

Old keyword in carb_coefs.cbn	New location
rtof	manure_c_frac column of carbon.bsn
cbn_consolidation_factors (2 values)	bio_consol, till_consol columns of carbon.bsn
cbn_factor_approaches (2 ints)	tmpf_eqn, watf_eqn columns of carbon.bsn
tn, top, tx	t_cbn_min, t_cbn_opt, t_cbn_max columns of carbon.bsn
zz_bmix_coefs (3 values)	bmix_a, bmix_b, bmix_c columns of carbon.bsn. zz_ prefix dropped to match the column name.
zz_emix_coefs (3 values)	tillmix_a, tillmix_b, tillmix_c columns of carbon.bsn
photo_degrade_factor	sfc_rsd_photodeg column of carbon.bsn
nmbr_soil_test_layers, soil_test	retired; no replacement (sub-table was unused downstream)

Five new tunables were promoted from hard-coded constants and added to carbon.bsn:

- n_act_frac (formerly the nactfr local in nut_nminrl.f90)
- cnr_cap, cnr_ref (formerly the 500 and 25 hard-coded values in cbn_surfrsd_decomp.f90)
- cpr_cap, cpr_ref (formerly the 5000 and 200 hard-coded values in cbn_surfrsd_decomp.f90)

Related files

- carbon.bsn. Replacement for the basin-wide scalars.
- carbon_lyr.bsn. Replacement for the per-layer table.
- codes.bsn. The cswat switch. cswat == 2 and cswat == 3 are no longer accepted; set 0 (no carbon) or 1 (active carbon).
- Carbon outputs. Replacement for the cbn_diagnostics flag.

Purpose

treatment.trt is a legacy file name carried over from the SWAT2012 treatment operation table. It is written by external setup tools (QSWAT+, SWAT+ Editor) into project directories, but no SWAT+ Fortran reader consumes it. The file plays no part in a SWAT+ run.

The current treatment functionality in SWAT+ lives in the water allocation module and uses water_treat.wal, not treatment.trt. See src/water_treatment_read.f90.

Source

A search of the SWAT+ Fortran tree at commit 8a1edd4a23915a2cac1cd90c788f0d521404700e finds no open, inquire, or filename string referencing treatment.trt or any .trt extension. file.cio does not list it.

```
grep -rn "treatment.trt\|\\.trt" swatplus/src/
# no matches
```

Format

There is no defined format. The file is not parsed.

Example

Both reference datasets ship an empty treatment.trt:

```
$ wc -l refdata/Ames_sub1/treatment.trt refdata/Osu_1hru/treatment.trt
0 refdata/Ames_sub1/treatment.trt
0 refdata/Osu_1hru/treatment.trt
```

The file exists only as a placeholder.

Related files

- water_treat.wal. The active treatment input, read by `water_treatment_read`. Defines water treatment objects with storage, lag, loss fraction, and constituent links used by water allocation.

Purpose

`manure_allo.mnu` defines manure-allocation objects. Each object names one or more source locations (manure storage on a farm, a feedlot, a lagoon) and one or more demand locations (HRUs, municipal treatment, inter-basin diversion). At runtime, manure produced at the sources is distributed to the demand objects under a named rule. Each allocation object holds its own source list, demand list, monthly production, storage, and per-source withdrawal history.

The allocation file ties three databases together:

- fertilizer.frt. Each source's `manure_typ` is a key into `fertdb` (cross-walked at read time into `src%fertdb`).
- lum.dtl. For HRU demands, the rule name is a key into `dtbl_lum` (cross-walked into `trn%dtbl_num`).
- `chem_app.ops`. The first action of the matched lum decision table names a chemical-application profile in `chemapp_db`, cross-walked into `manure_amt%app_method`.

Source

- Reader: src/manure_allocation_read.f90
- Types: src/manure_allocation_module.f90
  - `type manure_allocation` (variable `mallo`)
  - `type manure_source_objects` (`mallo(i)%src(j)`)
  - `type manure_demand_objects` (`mallo(i)%trn(j)`)

Format

- Line 1: title (skipped).
- Line 2: number of allocation objects, `imax`. Stored in `db_mx%mallo_db`.
- Per allocation object, in order:
  1. Header line (skipped).
  2. Object header record: `name`, `rule_typ`, `src_obs`, `trn_obs`.
  3. Header line (skipped).
  4. `src_obs` source records.
  5. Header line (skipped).
  6. `trn_obs` demand records.

Object header

#	Field	Type	Description
1	name	char(25)	Allocation-object name.
2	rule_typ	char(25)	Allocation rule. Selects how unmet demand is split across sources.
3	src_obs	int	Number of source objects in this allocation.
4	trn_obs	int	Number of demand objects in this allocation.

Source record (one per source)

#	Field	Type	Units	Description
1	num	int	none	Sequential source id. Used as the array index.
2	mois_typ	char(3)	none	Moisture class: `wet` or `dry`.

#	Field	Type	Units	Description
3	manure_typ	char(25)	none	Manure-product name. Cross-walked against `fertdb(:)%fertnm` and stored in `fertdb`.
4	lat	real	degrees	Latitude of the source.
5	long	real	degrees	Longitude of the source.
6	stor_init	real	t	Initial manure in storage.
7	stor_max	real	t	Storage capacity.
8..19	prod_mon(1..12)	real	t/month	Average monthly manure production. Twelve values, January through December.

Demand record (one per demand)

#	Field	Type	Description
1	num	int	Sequential demand id. Used as the array index.
2	ob_typ	char(10)	Receiver type: `hru` (apply to an HRU), `muni` (municipal treatment), or `divert` (inter-basin diversion).
3	ob_num	int	Number of the receiving object of `ob_typ`.
4	dtbl	char(25)	Decision-table name. For `hru`, matched against `dtbl_lum`.
5	right	char(2)	Allocation right. `sr` for senior, `jr` for junior.

For an HRU demand, the reader sets `hru(ihru)%man_trn_dtbl` to the resolved decision-table index, and looks up the first action's `option` against `chemapp_db(:)%name` to set the application method on `mallo(imro)%trn(itrn)%manure_amt%app_method`.

Example

A two-line skeleton with one source and one HRU demand:

```
manure_allo.mnu: example
1
name            rule_typ    src_obs  trn_obs
farm_a          fixed             1        1
num  mois_typ  manure_typ  lat       long      stor_init  stor_max   prod_mon(1..12)
  1  wet       beef_salt   41.99   -93.62          0.0       500.0    5.0  5.0  6.0  6.0  7.0  7.0  7.0  7
num  ob_typ    ob_num  dtbl                 right
  1  hru            1  apply_manure_dtbl       sr
```

The decision-table named `apply_manure_dtbl` must exist in `lum.dtl` and its first action must name an entry in `chem_app.ops` for the cross-walks to resolve.

Related files

- `fertilizer.frt`. Defines the manure-product names that source records reference, and the manure organic-matter database (`manure_db.frt`, `manure_om.frt`).
- `lum.dtl`. Decision tables that fire manure applications at each demand HRU.
- `chem_app.ops`. Chemical-application profiles (rate, method) selected by the decision-table action.

Five small inputs that the larger pages reference but do not document. Each is a single reader populating a single derived type.

management.sch

Fixed-schedule management. Each schedule lists scheduled operations on calendar dates or heat-unit fractions, plus a set of named auto-operation tables (decision tables in `lum.dtl`). `landuse.lum` selects a schedule by name through `mgt`.

- Reader: `src/mgt_read_mgtops.f90` (header) and `src/read_mgtops.f90` (operation rows).
- Default file name: `management.sch` (from `in_lum%management_sch` in `input_file_module.f90`).
- Target type: `management_schedule` in `src/mgt_operations_module.f90`, array `sched(:)`.

File format

- Line 1: title (skipped).
- Line 2: header (skipped).
- For each schedule:
  - One header row: `name`, `num_ops`, `num_autos`.
  - `num_autos` rows, each naming an auto-operation. If the name is `pl_hv_summer1` or `pl_hv_winter1` the row also lists one crop name; if it is `pl_hv_summer2` it lists two crop names.
  - `num_ops` operation rows, list-directed.

Schedule header

#	Field	Type	Description
1	name	char(40)	schedule name referenced by `landuse.lum`
2	num_ops	int	number of scheduled operation rows
3	num_autos	int	number of auto-operation tables

Operation row

Read into `management_ops` (see `mgt_operations_module.f90`).

#	Field	Type	Description
1	op	char(40)	operation code (see below)
2	mon	int	calendar month
3	day	int	calendar day of month
4	husc	real	heat-unit fraction trigger (0 if calendar-driven)
5	op_char	char(40)	name into the operation database keyed by `op`
6	op_plant	char(40)	name into a secondary database (plant, harvest type, chemical application)
7	op3	real	application amount or override value

Recognised `op` codes and the database that `op_char` (and where noted `op_plant`) resolves against:

Code	Action	op_char database	op_plant database
`pcom`	plant community init	`plant.ini` (`pcomdb`)	none
`plnt`	plant	`plants.plt` (`pldb`)	`transplant.plt` (`transpl`)
`harv`	harvest	(none)	`harv.ops` (`harvop_db`)
`hvkl`	harvest and kill	(none)	`harv.ops`
`kill`	kill	none	none
`till`	tillage	`tillage.til` (`tilldb`)	none
`irrm`, `irrp`	irrigation	`irr.ops` (`irrop_db`)	none
`fert`	fertilizer	`fertilizer.frt` (`fertdb`)	`chem_app.ops` (`chemapp_db`)
`manu`	manure	`manure_db.frt` (`manure_db`)	`chem_app.ops`
`pest`	pesticide	`pesticide.pes` (`pestdb`)	`chem_app.ops`
`graz`	grazing	`graze.ops` (`grazeop_db`)	none
`burn`	burn	`fire.ops` (`fire_db`)	none
`swep`	street sweep	`sweep.ops` (`sweepop_db`)	none
`skip`	end of rotation year	none	none
`prtp`	print plant variables	none	none

A `skip` row advances the internal rotation year counter; subsequent rows belong to the next year of the rotation.

Example

`refdata/Ames_sub1/management.sch`:

```
management.sch file AMES
                          NAME NUMB_OPS NUMB_AUTO OP_TYP   MON   DAY HU_SCH   OP_DATA1
 ↪  OP_DATA2  OP_DATA3
mgt_01                               305         1
                                                 irr_year_irr
                                                     till    4     1    0.0   chisplow
 ↪  null     0.000
                                                     plnt    4    28    0.0       corn
 ↪  null     0
                                                     irrm    5    15    0.0  sprinkler_ilm
 ↪  null     0.000
                                                     fert    6     1    0.0       null
 ↪  elem_n   181.400
                                                     hvkl   10     7    0.0       corn
 ↪  grain     0.000
                                                     skip    0     0    0         null
 ↪  null     0
```

Related files

- `landuse.lum` selects a schedule by name.
- `*.ops` hold the operation parameters that `op_char` and `op_plant` resolve against.
- `*.dtl` decision tables provide the `auto_name` entries.

puddle.ops

Rice paddy puddling parameters. A puddling event resets the upper soil-layer hydraulic conductivity and the in-paddy water chemistry to fixed values.

- Reader: `src/mgt_read_puddle.f90`.
- File name: hard-coded `puddle.ops`.
- Target type: `puddle_operation` in `src/mgt_operations_module.f90`, array `pudl_db(:)`.

File format

Line 1 title and line 2 header are skipped. Each remaining line is one record, list-directed.

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	wet_hc	real	0	mm/h	upper-layer hydraulic conductivity after puddling
3	sed	real	0	ppm	sediment concentration after puddling
4	orgn	real	0	ppm	organic N concentration after puddling
5	sedp	real	0	ppm	organic P concentration after puddling
6	no3	real	0	ppm	NO3-N concentration after puddling
7	solp	real	0	ppm	mineral (soluble) P concentration after puddling
8	nh3	real	0	ppm	NH3 concentration after puddling
9	no2	real	0	ppm	NO2 concentration after puddling

Related files

- `*.ops` for the other operation databases that follow the same skip-two-and-read pattern.

transplant.plt

Plant transplant database. Initial state for a plant that is established by transplant rather than from seed. Operation rows with `op = plnt` look up `op_plant` against this database.

- Reader: `src/plant_transplant_read.f90`.
- File name: hard-coded `transplant.plt`.
- Target type: `plant_transplant_db` in `src/plant_data_module.f90`, array `transpl(:)`.

File format

Line 1 title and line 2 header are skipped. Each remaining line is one record.

#	Field	Type	Default	Units	Description
1	name	char(40)	`frsd`	none	transplant entry name
2	lai	real	0	$m^{2/m}2$	initial leaf area index
3	bioms	real	0	kg/ha	initial biomass
4	phuacc	real	0	fraction	initial fraction of accumulated heat units
5	fr_yrmat	real	0.05	fraction	fraction of current year of growth to years to maturity
6	pop	real	0	plants/m^2	initial plant population

Related files

- plants.plt for the underlying plant parameter database.
- management.sch resolves the op_plant field against transplant.plt for plnt operations.

co2.out and co2_yr.dat

CO2 driver. Despite the .out extension, co2_read is the routine that establishes the per-year atmospheric CO2 series used by the simulation. The reader reads co2_yr.dat if present, fills the per-year array co2y(:), and writes the resolved series to co2.out as a record of what was used.

- Reader: src/co2_read.f90.
- Input file (optional): co2_yr.dat.
- Output file: co2.out (unit 2222).
- Fallback: when co2_yr.dat is missing, every simulation year is filled with bsn_prm%co2 from parameters.bsn.

co2_yr.dat format

Line	Content
1	title (skipped)
2	integer yrs, the number of year records that follow
3	header (skipped)
4 to 3+yrs	one record per year: iyr (int, calendar year) and co2 (real, ppm)

How the series is built

For a simulation that runs time%nbyr years starting in time%yrc_start:

- If co2_yr.dat is absent or empty, every simulation year is set to bsn_prm%co2.
- If the latest year in co2_yr.dat is at or before the simulation start, every simulation year is set to that final value.
- Otherwise the file is aligned to the simulation start. Simulation years that fall before the first record use the first record's value; years that fall after the last record carry the last value forward.

co2.out format

Header YR CO2(ppm), then one row per simulation year with the calendar year and the resolved ppm value.

Related files

- codes.bsn and parameters.bsn for the co2 fallback in bsn_prm.

object.prt

Per-object output selector. Each row asks the model to write one specific hydrograph or state variable for one spatial object to a named file. Reuses the old saveconc mechanism.

- Reader: src/object_read_output.f90.
- Default file name: object.prt (from in_sim%object_prt in input_file_module.f90).
- Target type: object_output in src/hydrograph_module.f90, array ob_out(:).

File format

Line 1 title and line 2 header are skipped. Each remaining row is one selection.

#	Field	Type	Description
1	k	int	row id (the reader also uses it to size the array)
2	obtyp	char(10)	object type (see below)
3	obtypno	int	object number within `obtyp` (1-based)
4	hydtyp	char(20)	hydrograph or state code (see below)
5	filename	char(26)	output file name to open and write to

Object types recognised by the reader:

Code	Object
`hru`	HRU
`hlt`	HRU LTE
`ru`	routing unit
`res`	reservoir
`cha`	channel
`exc`	export coefficient object
`dr`	delivery ratio object
`out`	outlet
`sdc`	SWAT-Deg channel

Hydrograph or state codes:

Code	Content	Header written
`tot`	total flow	`hyd_hdr_time hyd_hdr`
`rhg`	recharge	`hyd_hdr_time hyd_hdr`
`sur`	surface	`hyd_hdr_time hyd_hdr`
`lat`	lateral	`hyd_hdr_time hyd_hdr`
`til`	tile	`hyd_hdr_time hyd_hdr`
`sol_water`	soil moisture by layer	`hyd_hdr_time sol_hdr`
`solnut_ly`	soil N and P by layer	(no header)
`solnut_pr`	soil N and P for profile	(no header)
`plant`	plant status	`hyd_hdr_time plt_hdr plt_hdr`
`cha_fp`	channel and flood-plain water balance	`hyd_hdr_time fp_hdr`

Each row opens its own file at unit `6100 + i` and writes the matching header. The simulation appends to that file as the run progresses.

Related files

- `print.prt` controls the main output selection. `object.prt` is the side channel for per-object hydrograph files.